边界 3：input_dev——类驱动与输入核心之间. 类驱动解析完 HID 报告后，只调用 input_report_key()/input_sync() 把语义化事件丢进输入核心，完全不知道 evdev、控制台或任何具体消费者。输入核心再把它分发给每个打开的 evdev_client（图里最上面那条边界），同时另一路经 consoleintr() 复用串口 TTY 的回显与行规程——一份报告、两个消费者。

正交的一层：通用设备模型. 上面五层是“数据通路”，还有一层横向的注册/绑定机制把它们接起来，即 device.c 的 struct bus_type。USB core 向设备模型注册一条名为 usb 的总线（usb_bus，带 match/probe/remove/modalias 四个钩子），类驱动 usb_register_driver()、枚举出的设备 usb_device_register() 分别把自己挂上去；一旦设备和驱动在总线上相遇，usb_match() 按 class/subclass/protocol 或 ID 表判定匹配，usb_probe() 调用类驱动的 probe()。热插拔与自动装载正来自这一层：匹配失败时 modalias 生成 usb:cXXsXXpXX 触发 request_module() 去加载对应的 .ko（详见 §7.4.9 与鼠标自动装载一节）。

与 Linux 的对应与简化. 表 7.2 给出每层对应的 Linux 路径。本项目有两处刻意的简化：其一，没有独立的 usb_hcd 对象，也没有单独的 hub-driver 层——dwc2.c 同时充当 platform 驱动（HCD）、根 Hub 的模拟者（dwc2_root_hub 复用自己的 dwc2_usb_ops）和总线枚举者；其二，usb_host_ops 把异步 URB 接口和遗留的同步 control/bulk 混在同一张表里，类驱动在枚举阶段可以直接调用同步接口，运行期才走 URB。

表 7.2: 本项目 USB 软件栈各层与 Linux 的对应

本项目	Linux 对应	职责
dwc2.c（usb_host_ops）	drivers/usb/host/dwc2 + usb_hcd	把 URB 翻译成 host channel 寄存器与 DMA，收完成中断
usb.c	drivers/usb/core	URB 生命周期、giveback、设备/驱动注册入口
device.c（bus_type）	drivers/base	总线/设备/驱动模型，match–probe、热插拔、自动装载
usbkbd.c、mt7601u.c	drivers/hid/usbhid、drivers/net/usb	类驱动：解析各自协议的报告/报文
input.c + evdev	drivers/input + drivers/input/evdev	事件分发到 /dev/input/eventN

7.4.3 事务格式：Token、Data、Handshake 三相

上一小节说“主机发起一切”，但一次到底“发起”了什么？USB 总线上被独立调度的最小单位既不是字节也不是包，而是**事务**（transaction）。一次事务由主机发起，按固定顺序串起最多三种**包**（packet）：主机先发一个 **Token** 包，声明“要对哪个设备的哪个端点做一次什么操作”；接着是一个可选的 **Data** 包，方向取决于 Token 的类型；最后是一个 **Handshake** 包，由接收方确认或拒绝。规范里的图 8-30（bulk 事务）和图 8-36（control SETUP 事务）画的正是这套三相结构在不同传输类型下走过的分支。

每个包的场（field）布局是固定的：以 SYNC 开场让接收方对齐位边界，紧跟一个 PID 场标明包的类型，中间是地址/负载与校验，末尾以 EOP（end of packet）收束，如图 7.5。

PID 决定了一个包在事务里扮演什么角色，表 7.3 按 Token/Data/Handshake 三类把本项目会遇到的 PID 列全。

三相结构在三种传输类型下走的路径略有不同，图 7.6 把控制 SETUP、IN、OUT 三条主线并排画出（框的颜色标出这个包由谁发出：灰框 = 主机、白框 = 设备）。

把这套三相格式对回本项目的三个 channel 就很清楚了：ch0 的枚举是反复做 control 事务——先 SETUP 三相把“读描述符/设配置”的请求送进去，再用若干 IN 或 OUT 事务搬运数据，最后用一个反方向的空 Data 事务做 STATUS；ch6 键盘每隔 bInterval 发一次 IN 事务，设备没按键就回 NAK、有按键才回 DATA1/DATA0；ch4 MT7601U 的 bulk-IN 同样发 IN，靠 DATA0/DATA1 的翻转做流控。下一小节的 split 事务，本质就是把这里的三相包在高速 Hub 的事务转换器（TT）处拆成两拍：主机先发一个 SSPLIT 触发 TT 去总线上跑完整事务，隔 1 ~ 2 个微帧再发 CSPLIT 把结果（DATA、ACK、NAK 或 NYET）取回来。

规范按“可靠性、时限、方向”把这同一套三相事务组合成**四种传输类型**（表 7.4）。前三者都靠 Handshake 与 DATA0/DATA1 翻转保证不丢不错，只有 isochronous 干脆去掉握手、发出去就不管，用可靠性换实时性；bulk